院赛作品展示与AI工具导航平台 — 全栈开发指令

一、项目总体说明

请帮我从零搭建一个完整的前后端分离 Web 应用，项目名称为 **院赛作品展示与AI工具导航平台**。

该平台面向学校院系，核心功能如下：

1. **学生参赛作品展示**：按学年、学期、专业多维度浏览优秀学生作品
2. **AI工具导航**：分类展示推荐的AI工具，提供官网直达链接
3. **新闻资讯发布**：发布赛事新闻稿、活动通知
4. **两级用户体系**：管理员（全权）+ 教师（发布本人内容）

二、技术栈（严格遵守）

层次	技术	版本要求
前端框架	Vue 3 + Vite	Vue >= 3.4，使用 `<script setup>` 语法
UI 组件库	Element Plus	>= 2.7
前端路由	Vue Router 4	使用 history 模式
前端状态	Pinia	>= 2.0
HTTP 客户端	Axios	统一封装请求拦截器
富文本/Markdown编辑器	md-editor-v3	支持 Markdown 编写与实时预览
后端框架	Python FastAPI	>= 0.110
ORM	SQLAlchemy 2.x	async 模式
数据库	MySQL 8	
数据库迁移	Alembic	
密码哈希	passlib[bcrypt]	
认证	JWT（python-jose）	access_token 有效期 8h
文件存储	本地磁盘	存于 `backend/uploads/` 目录，通过接口提供静态访问

层次	技术	版本要求
跨域处理	FastAPI CORS 中间件	开发环境允许 localhost:5173

三、项目目录结构

请严格按以下结构初始化项目：

```
project-root/
├── frontend/                   # Vue 3 前端
|   ├── src/
|   |   ├── api/                # Axios 请求封装，按模块分文件
|   |   ├── assets/
|   |   ├── components/         # 通用组件
|   |   |   ├── common/         # 全局通用（AppHeader，AppFooter等）
|   |   |   └── business/       # 业务组件（WorkCard，NewsCard等）
|   |   ├── router/             # index.js，含路由守卫
|   |   ├── stores/             # Pinia stores（auth，works，news，tools）
|   |   ├── views/
|   |   |   ├── public/         # 公开页（首页、作品列表、详情、新闻、AI工具）
|   |   |   ├── auth/           # 登录页
|   |   |   ├── teacher/        # 教师中心（个人中心、我的作品、我的新闻）
|   |   |   └── admin/          # 管理后台（仪表盘、用户管理、内容管理）
|   |   ├── utils/              # 工具函数（request.js，format.js等）
|   |   └── main.js
|   ├── vite.config.js          # 配置代理到后端 8000 端口
|   └── package.json
|
├── backend/                    # FastAPI 后端
|   ├── app/
|   |   ├── api/                # 路由层，按模块分文件
|   |   |   ├── auth.py
|   |   |   ├── works.py
|   |   |   ├── news.py
|   |   |   ├── ai_tools.py
|   |   |   ├── users.py
|   |   |   └── upload.py
|   |   ├── core/
|   |   |   ├── config.py       # 配置（从.env读取）
|   |   |   ├── security.py     # JWT、密码工具
|   |   |   └── deps.py         # FastAPI 依赖注入（get_db，get_current_user等）
|   |   ├── models/             # SQLAlchemy 模型
|   |   ├── schemas/            # Pydantic schemas（请求/响应体）
|   |   ├── crud/               # 数据库操作函数
|   |   └── main.py             # FastAPI 应用入口，注册路由
|   ├── alembic/                # 数据库迁移
|   ├── uploads/                # 本地上传文件存储目录
|   ├── requirements.txt
|   └── .env.example            # 环境变量示例
|
└── README.md                   # 启动说明
```

四、数据库设计

4.1 建表要求

使用 SQLAlchemy 2.x 定义以下模型，使用 Alembic 生成迁移脚本。

users 表

```sql
id            INT PK AUTO_INCREMENT
username      VARCHAR(50) UNIQUE NOT NULL        -- 工号/账号
password_hash VARCHAR(255) NOT NULL
role          ENUM('admin','teacher') NOT NULL
name          VARCHAR(50) NOT NULL               -- 真实姓名
department    VARCHAR(100)                       -- 所属院系
avatar_url    VARCHAR(500)
status        TINYINT DEFAULT 1                  -- 1启用 0禁用
created_at    DATETIME DEFAULT NOW()
updated_at    DATETIME ON UPDATE NOW()
```

majors 表（专业）

```sql
id         INT PK AUTO_INCREMENT
name       VARCHAR(100) NOT NULL
sort_order INT DEFAULT 0
```

works 表（学生作品）

```sql
```

```
id              INT PK AUTO_INCREMENT
title           VARCHAR(100) NOT NULL
author_names    VARCHAR(200) NOT NULL              -- 参赛学生姓名，逗号分隔
guide_teacher   VARCHAR(100)                       -- 指导教师姓名
major_id        INT FK -> majors.id
academic_year   VARCHAR(20) NOT NULL               -- 如 2024-2025
semester        TINYINT NOT NULL                   -- 1上学期 2下学期
contest_name    VARCHAR(200)                       -- 赛事名称
award           VARCHAR(50)                        -- 获奖情况
cover_url       VARCHAR(500)
content         LONGTEXT                           -- Markdown 内容
demo_url        VARCHAR(500)                       -- 演示链接
attachment_url  VARCHAR(500)                       -- 附件链接
status          ENUM('draft','published','archived') DEFAULT 'draft'
publisher_id    INT FK -> users.id
published_at    DATETIME
created_at      DATETIME DEFAULT NOW()
updated_at      DATETIME ON UPDATE NOW()
```

news 表（新闻稿）

```sql
id           INT PK AUTO_INCREMENT
title        VARCHAR(200) NOT NULL
content      LONGTEXT                              -- Markdown 内容
cover_url    VARCHAR(500)
category     ENUM('contest','notice','honor','other') DEFAULT 'contest'
author_id    INT FK -> users.id
status       ENUM('draft','published') DEFAULT 'draft'
is_top       TINYINT DEFAULT 0                     -- 是否置顶
published_at DATETIME
created_at   DATETIME DEFAULT NOW()
updated_at   DATETIME ON UPDATE NOW()
```

ai_categories 表（AI工具分类）

```sql
id          INT PK AUTO_INCREMENT
name        VARCHAR(50) NOT NULL
icon        VARCHAR(50)                           -- Element Plus 图标名
sort_order  INT DEFAULT 0
```

ai_tools 表（AI工具）

```sql
id          INT PK AUTO_INCREMENT
name        VARCHAR(100) NOT NULL
logo_url    VARCHAR(500)
url         VARCHAR(500) NOT NULL                -- 官网链接
description VARCHAR(300)
category_id INT FK -> ai_categories.id
is_free     TINYINT DEFAULT 1                    -- 1免费 0付费
rating      TINYINT DEFAULT 5                    -- 1-5星
is_featured TINYINT DEFAULT 0                    -- 是否精选（访客可见）
sort_order  INT DEFAULT 0
created_at  DATETIME DEFAULT NOW()
```

operation_logs 表（操作日志）

```sql
id          INT PK AUTO_INCREMENT
user_id     INT FK -> users.id
action      VARCHAR(100) NOT NULL                -- 如 create_work, delete_news
target_type VARCHAR(50)                          -- 如 work, news, user
target_id   INT
detail      TEXT
created_at  DATETIME DEFAULT NOW()
```

4.2 初始数据

应用首次启动时，通过启动脚本自动插入：

- 一个默认管理员账号：username=`admin`，password=`Admin@123456`，name=`超级管理员`
- 8条 ai_categories 初始分类：编程开发、图像生成与处理、写作与文案、视频制作、数据分析、演示文稿、语音与音频、综合大模型
- 5条 majors 初始数据：计算机科学与技术、软件工程、数字媒体技术、人工智能、网络工程

五、后端 API 设计

所有接口前缀：`/api/v1` 响应格式统一为：

```json
```

```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

错误时 `code` 为对应 HTTP 状态码，`message` 为错误说明。

5.1 认证接口 `/api/v1/auth`

方法	路径	说明	权限
POST	`/login`	登录，返回 access_token	公开
GET	`/me`	获取当前用户信息	登录
PUT	`/me`	修改个人信息	登录
PUT	`/me/password`	修改密码	登录

5.2 作品接口 `/api/v1/works`

方法	路径	说明	权限
GET	`/`	作品列表（分页+筛选）	公开（仅 published）
GET	`/{id}`	作品详情	公开（仅 published）
POST	`/`	新建作品	教师/管理员
PUT	`/{id}`	编辑作品	本人或管理员
DELETE	`/{id}`	删除作品	本人（草稿）或管理员
PUT	`/{id}/publish`	发布作品	本人或管理员
PUT	`/{id}/archive`	下架作品	管理员
GET	`/my`	我发布的作品	教师/管理员
GET	`/admin/all`	所有作品（含草稿）	管理员

GET `/` 查询参数：

- `page`, `page_size`（默认12）
- `academic_year`, `semester`, `major_id`, `keyword`（模糊搜索标题/作者名）

- `sort`：`created_at_desc`（默认）、`title_asc`

5.3 新闻接口 `/api/v1/news`

方法	路径	说明	权限
GET	`/`	新闻列表（分页）	公开（仅 published）
GET	`/{id}`	新闻详情	公开（仅 published）
POST	`/`	新建新闻	教师/管理员
PUT	`/{id}`	编辑新闻	本人或管理员
DELETE	`/{id}`	删除新闻	本人（草稿）或管理员
PUT	`/{id}/publish`	发布新闻	本人或管理员
PUT	`/{id}/top`	置顶/取消置顶	管理员
GET	`/my`	我发布的新闻	教师/管理员

GET `/` 查询参数：`page`, `page_size`（默认10）, `category`, `keyword`

5.4 AI工具接口 `/api/v1/ai-tools`

方法	路径	说明	权限
GET	`/categories`	获取所有分类	公开
GET	`/`	工具列表（按分类）	登录用户全量；未登录仅返回 is_featured=1 的条目
POST	`/`	新建工具	管理员
PUT	`/{id}`	编辑工具	管理员
DELETE	`/{id}`	删除工具	管理员
POST	`/categories`	新建分类	管理员
PUT	`/categories/{id}`	编辑分类	管理员
DELETE	`/categories/{id}`	删除分类	管理员

5.5 用户管理接口 `/api/v1/users`（仅管理员）

方法	路径	说明
GET	`/`	教师列表（分页）
POST	`/`	创建教师账号
PUT	`/{id}`	编辑教师信息
PUT	`/{id}/status`	启用/禁用账号
PUT	`/{id}/reset-password`	重置密码

5.6 文件上传接口 `/api/v1/upload`

方法	路径	说明	权限
POST	`/image`	上传图片（封面图、Logo等）	登录
POST	`/file`	上传附件（PDF/ZIP，最大50MB）	教师/管理员

- 图片上传：支持 JPG/PNG/WebP，最大 5MB，存至 `uploads/images/`，返回访问 URL
- 文件存储命名规则：`{年月日}_{uuid[:8]}_{原始文件名}`
- 通过 FastAPI `StaticFiles` 挂载 `/uploads` 路径提供静态文件访问

5.7 其他公共接口

方法	路径	说明	权限
GET	`/api/v1/majors`	专业列表	公开
GET	`/api/v1/stats`	平台统计（作品数/新闻数/工具数）	公开
GET	`/api/v1/logs`	操作日志列表	管理员

六、前端页面详细需求

6.1 路由结构

```javascript
```

```
// 公开路由
/                          -> HomeView          首页
/works                     -> WorksView         作品列表
/works/:id                 -> WorkDetailView    作品详情
/news                      -> NewsView          新闻列表
/news/:id                  -> NewsDetailView    新闻详情
/ai-tools                  -> AiToolsView       AI工具导航

// 认证
/login                     -> LoginView         登录页

// 教师中心（需登录）
/profile                   -> ProfileView       个人资料
/my/works                  -> MyWorksView       我的作品
/my/works/create           -> WorkFormView      新建作品
/my/works/:id/edit         -> WorkFormView      编辑作品
/my/news                   -> MyNewsView        我的新闻
/my/news/create            -> NewsFormView      新建新闻
/my/news/:id/edit          -> NewsFormView      编辑新闻

// 管理后台（需管理员角色，路由守卫拦截）
/admin                     -> AdminDashboard    仪表盘（统计数据）
/admin/works               -> AdminWorksView    作品管理
/admin/news                -> AdminNewsView     新闻管理
/admin/ai-tools            -> AdminAiToolsView  AI工具管理
/admin/users               -> AdminUsersView    用户管理
/admin/logs                -> AdminLogsView     操作日志
```

路由守卫规则：

- `/my/*` 和 `/admin/*`：未登录跳转 `/login`
- `/admin/*`：非管理员角色返回 403 页面

6.2 全局布局

公开页面（HomeView、WorksView等）：

- 使用 `AppLayout` 组件，包含顶部导航 `AppHeader` + 主内容区 + `AppFooter`
- `AppHeader`：左侧 Logo（学院名+平台名），导航菜单（首页/作品展示/AI工具/新闻资讯），右侧登录状态（未登录显示"登录"按钮，已登录显示头像下拉菜单）
- `AppFooter`：版权信息

管理后台页面：

- 使用 Element Plus `ElContainer` 布局：左侧固定侧边栏（`ElMenu` 竖排导航） + 右侧内容区

- 侧边栏宽度 220px，含 Logo + 各管理菜单项，底部显示当前登录用户信息

6.3 各页面详细要求

首页（HomeView）

- **Hero 区**：大标题 + 副标题 + "浏览作品"按钮
- **平台统计条**：展示3个数字（作品总数、新闻总数、AI工具数），调用 `/api/v1/stats`
- **最新作品**：横向卡片轮播或 Grid 展示最新6条已发布作品，每张卡片含封面图、作品名、专业、获奖情况、学年信息
- **最新新闻**：列表展示最近4条新闻，含标题、摘要（正文前80字）、发布时间
- **快速入口**：4个图标按钮快速跳转各主要功能页

作品列表页（WorksView）

- 顶部筛选栏：学年（`ElSelect`）、学期（`ElSelect`）、专业（`ElSelect` 多选）、关键词搜索框（`ElInput`）+ 重置按钮
- 视图切换：右上角 Grid/List 图标切换
- **Grid视图**：每行3列卡片（`WorkCard` 组件），卡片含：封面图（16:9）、作品名（加粗，最多2行省略）、专业标签、获奖徽章（按获奖等级用不同颜色 `ElTag`）、参赛学生名
- **List视图**：`ElTable` 展示，列：封面缩略图、作品名、专业、学年学期、获奖情况、发布时间
- 底部 `ElPagination`，每页12条（Grid）/ 20条（List）

作品详情页（WorkDetailView）

- 左侧主区（占宽60%）：
  - 封面大图
  - 作品名称（H1）
  - 标签行：专业、学年学期、获奖标签
  - 作品简介（Markdown 渲染，使用 `md-editor-v3` 的预览模式）
- 右侧信息栏（占宽40%）：
  - 参赛信息卡：参赛学生、指导教师、赛事名称
  - 操作区：演示链接按钮（若有）、下载附件按钮（若有）
  - 发布信息：发布教师、发布时间

AI工具页（AiToolsView）

- 顶部说明：已登录显示全量工具；未登录显示提示横幅"登录后查看全部工具"
- 按分类展示，每个分类一个标题 + 工具卡片横向滚动行（或换行Grid）

- 每个工具卡片：Logo + 工具名 + 简介（最多2行省略）+ 免费/付费 `ElTag` + ★评分 + "立即使用"按钮（外链跳转官网）
- 访客只看到各分类中 `is_featured=1` 的工具，其余用模糊遮罩 + 登录提示替代

新闻列表页（NewsView）

- 顶部分类筛选 `ElTabs`：全部 / 赛事报道 / 活动通知 / 荣誉展示
- 置顶新闻：若有 `is_top=1` 的新闻，用带"置顶"标记的卡片单独展示在顶部
- 新闻卡片：封面图（左）+ 标题 + 摘要 + 作者 + 发布时间（右）
- `ElPagination`，每页10条

新闻详情页（NewsDetailView）

- 标题（H1）+ 元信息（作者、发布时间、分类）
- Markdown 正文渲染（md-editor-v3 预览模式）
- 底部：复制链接按钮

登录页（LoginView）

- 居中卡片布局（宽400px）
- `ElForm`：账号（工号）、密码输入框 + 登录按钮
- 登录成功后根据角色跳转：管理员 → `/admin`，教师 → `/profile`

作品发布/编辑表单（WorkFormView）

- `ElForm` 布局，字段：
  - 作品名称（`ElInput`，必填）
  - 参赛学生姓名（`ElInput`，必填，说明用逗号分隔）
  - 指导教师（`ElInput`）
  - 专业（`ElSelect`，从接口加载专业列表，必填）
  - 班级（`ElInput`）
  - 学年（`ElSelect`，必填；选项动态生成近5学年）
  - 学期（`ElRadioGroup`：上学期/下学期，必填）
  - 赛事名称（`ElInput`）
  - 获奖情况（`ElSelect`：一等奖/二等奖/三等奖/优秀奖/参与奖/无）
  - 封面图（`ElUpload`，调用 `/api/v1/upload/image`，上传后显示预览，必填）
  - 作品简介（`md-editor-v3` 编辑器，必填）
  - 演示链接（`ElInput`，URL格式校验）
  - 附件（`ElUpload`，调用 `/api/v1/upload/file`）

- 底部按钮：保存草稿 + 直接发布（两个按钮，分别提交不同 status）

新闻发布/编辑表单（NewsFormView）

- 字段：标题（必填）、分类（`ElSelect`）、封面图（`ElUpload`）、正文（`md-editor-v3`）
- 底部：保存草稿 + 发布

个人中心（ProfileView）

- 头像上传 + 修改基本信息表单
- 修改密码区块（旧密码 + 新密码 + 确认新密码）

管理后台 - 仪表盘（AdminDashboard）

- 统计卡片4个：作品总数、新闻总数、AI工具数、教师账号数
- 图表：近6个月发布作品趋势（折线图，使用 Element Plus 的 ECharts 或自行用 Canvas 简单实现）
- 最近操作日志（最新10条）

管理后台 - 作品管理（AdminWorksView）

- `ElTable` + `ElPagination`，列：封面缩略图、作品名、专业、学年学期、状态（`ElTag`）、发布者、操作（编辑/发布/下架/删除）
- 顶部筛选：状态、学年、关键词
- 支持批量操作（`ElTable` 多选）：批量发布、批量下架

管理后台 - 用户管理（AdminUsersView）

- `ElTable` 列：姓名、工号、院系、状态、创建时间、操作（编辑/启用禁用/重置密码）
- 新建教师按钮 → `ElDialog` 弹出表单（姓名、工号、初始密码、院系）

管理后台 - AI工具管理（AdminAiToolsView）

- 上半区：分类管理（简单列表 + 新增/编辑/删除分类）
- 下半区：工具列表表格 + 新增/编辑（`ElDialog` 弹窗表单）/删除

七、通用前端开发要求

7.1 Axios 封装

`src/utils/request.js` 统一封装：

- baseURL 读取环境变量 `VITE_API_BASE_URL`（开发时通过 Vite proxy 转发，避免跨域）
- 请求拦截器：自动在 Header 中加入 `Authorization: Bearer {token}`（token 从 Pinia auth store 获取）
- 响应拦截器：
  - `code === 200` 正常返回 `data`
  - `code === 401` 清除 token，跳转 `/login`
  - 其他错误统一用 `ElMessage.error` 展示 `message`

7.2 Pinia 状态

- `useAuthStore`：存储 `token`、`userInfo`，持久化到 `localStorage`
- `useWorksStore`：作品列表缓存（可选）
- 各 store 提供对应 action 调用 api 方法

7.3 Vite 代理配置

```javascript
// vite.config.js
server: {
  proxy: {
    '/api': { target: 'http://localhost:8000', changeOrigin: true }
  }
}
```

7.4 UI 规范

- 操作反馈：所有提交按钮点击后显示 `loading` 状态，成功/失败均用 `ElMessage` 提示
- 删除操作：必须用 `ElMessageBox.confirm` 二次确认
- 空状态：列表无数据时显示 Element Plus `ElEmpty` 组件
- 图片加载失败时显示占位符图片
- 表单校验：所有必填字段均加 `required` 规则，提交前校验

八、后端开发要求

8.1 安全

- 所有需要权限的接口使用 `Depends(get_current_user)` 注入
- 编写 `require_admin`、`require_teacher_or_admin` 装饰器/依赖
- 教师只能操作自己发布的内容（`publisher_id == current_user.id`），管理员不受限

- 密码最少8位，包含字母和数字

8.2 分页

所有列表接口统一分页格式，响应 `data` 包含：

```json
{
  "items": [],
  "total": 100,
  "page": 1,
  "page_size": 12,
  "pages": 9
}
```

8.3 操作日志

以下操作需自动写入 `operation_logs`：

- 发布/下架/删除作品
- 发布/删除新闻
- 创建/禁用/删除用户账号
- 新增/删除AI工具

8.4 环境变量（.env）

```
DATABASE_URL=mysql+asyncmy://root:password@localhost:3306/contest_platform
SECRET_KEY=your-secret-key-change-in-production
UPLOAD_DIR=./uploads
MAX_IMAGE_SIZE=5242880
MAX_FILE_SIZE=52428800
```

九、初始化与启动脚本

前端启动

```bash
cd frontend
npm install
npm run dev   # 开发服务器 localhost:5173
```

后端启动

```bash
cd backend
pip install -r requirements.txt
# 配置 .env 文件
alembic upgrade head          # 执行数据库迁移
python -m app.init_db          # 写入初始数据（admin账号、分类、专业等）
uvicorn app.main:app --reload --port 8000
```

README.md 要求

包含完整的本地开发启动步骤，包括 MySQL 数据库创建命令：

```sql
CREATE DATABASE contest_platform CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

十、开发顺序建议（按此顺序实现）

1. **后端基础框架**：FastAPI 应用骨架、数据库连接、模型定义、Alembic 迁移
2. **认证模块**：注册（仅管理员创建教师）、登录、JWT、权限依赖
3. **文件上传接口**：图片+附件上传，静态文件服务
4. **作品CRUD接口**（含分页、筛选）
5. **新闻CRUD接口**
6. **AI工具接口**
7. **用户管理接口**
8. **前端基础结构**：Vue Router、Pinia、Axios封装、布局组件
9. **公开页面**：首页、作品列表/详情、新闻列表/详情、AI工具页
10. **认证页面**：登录页
11. **教师功能**：个人中心、作品发布/管理、新闻发布/管理
12. **管理员后台**：全部管理页面

十一、附加要求

- 代码中的注释用**中文**书写
- 所有用户可见的提示文字（Element Plus 的 placeholder、提示等）用**中文**
- 后端接口文档通过 FastAPI 自动生成（`/docs`），无需额外编写
- 前端生产构建命令：`npm run build`，产物在 `frontend/dist/`

- 禁止使用任何付费或需要账号才能访问的第三方服务
- 所有功能在本地局域网环境下可完整运行，不依赖外部 CDN（除开发依赖外）

请开始按上述要求完整实现该项目，从后端基础框架开始，逐步完成所有功能模块。遇到需要决策的技术细节，请按最佳实践选择，无需询问。